SQL avançado e otimização

JANELAS

Ranking por grupo

```
SELECT categoria, valor,
 ROW_NUMBER() OVER (
 PARTITION BY categoria
 ORDER BY valor DESC) AS pos
FROM vendas;
```

Empate: defina critério estável.

Acumulado

```
SELECT data,valor,
 SUM(valor) OVER(ORDER BY data)
 AS acumulado FROM vendas;
```

ORDER BY precisa ser determinístico.

CTE / DIAGNÓSTICO

CTE

```
WITH totais AS (
 SELECT categoria,SUM(valor) AS total
 FROM vendas GROUP BY categoria)
SELECT * FROM totais WHERE total>500;
```

CTE facilita leitura.

Plano PostgreSQL

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM vendas WHERE id=42;
```

ANALYZE executa consulta!

Índice

```
CREATE INDEX idx_vendas_categoria
ON vendas(categoria);
```

Índices custam espaço/escrita.

INTEGRIDADE

Upsert PostgreSQL

```
INSERT INTO itens(id,valor)
VALUES(1,10) ON CONFLICT(id)
DO UPDATE SET valor=EXCLUDED.valor;
```

Requer restrição UNIQUE/PK.

Subconsulta

```
SELECT * FROM clientes c WHERE EXISTS
(SELECT 1 FROM pedidos p
 WHERE p.cliente_id=c.id);
```

EXISTS não duplica cliente.

Checar duplicatas

```
SELECT id,COUNT(*) FROM vendas
GROUP BY id HAVING COUNT(*)>1;
```

Cardinalidade importa.

Receitas de consulta: adapte dados, colunas, credenciais e versões. Execute somente em ambientes autorizados.